

Документация проекта GraphRAGon с поддержкой Graph of Thoughts

Utopialvo

27 июня 2026 г.

Содержание

1	Введение	2
2	Архитектура системы	2
3	Модель графа знаний	3
4	Обработка входного текста	4
5	Разрешение конфликтов	5
5.1	Обнаружение конфликтов	5
5.2	Итеративное разрешение	6
6	Вопросно-ответная система	6
6.1	Простой режим (<code>use_got=False</code>)	7
6.2	Graph of Thoughts (<code>use_got=True</code>)	7
7	Хранилище Memgraph	8
7.1	Транзакции и блокировки	8
7.2	Работа с пассажами	8
7.3	Отношения и <code>based_on</code>	8
7.4	Векторный поиск	9
7.5	Снепшоты и очистка	9
8	Клиенты LLM и эмбедингов	9
8.1	LLMClient	9
8.2	EmbeddingClient	10
9	Утилиты безопасного парсинга JSON	10
10	Особенности реализации и принятые решения	11
11	Заключение	12

1 Введение

Graph RAG (Retrieval-Augmented Generation) — это подход к вопросно-ответным системам, объединяющий поиск релевантной информации в графе знаний и генерацию ответа с помощью большой языковой модели (LLM). В отличие от классических RAG-систем, опирающихся на векторный поиск по коллекции неструктурированных документов, графовая версия хранит извлечённые факты в виде сущностей и связей между ними. Это позволяет моделировать семантические отношения и давать более точные, контекстные ответы. Под графом знаний здесь понимается структура, состоящая из узлов-сущностей и рёбер-отношений, каждое из которых снабжено указанием на исходный текстовый фрагмент (пассаж), обеспечивая прослеживаемость (provenance). Векторные представления (эмбединги) сущностей, полученные с помощью модели sentence-transformers, используются для быстрого поиска семантически близких узлов через встроенный в графовую СУБД Memgraph векторный индекс.

Данный проект реализует полный пайплайн:

- Загрузка произвольного текста, его нарезка на пересекающиеся чанки.
- Извлечение именованных сущностей и связей из каждого чанка с помощью локальной LLM.
- Сохранение сущностей, связей и исходных фрагментов в графовой СУБД Memgraph.
- Разрешение конфликтов между противоречащими друг другу отношениями (опциональный этап, запускается отдельным вызовом).
- Ответ на пользовательский вопрос с использованием техники *Graph of Thoughts* (GoT): построение гибкого графа операций (*Graph of Operations*), включающего генерацию, оценку, селекцию, итеративное улучшение и слияние мыслей. Это позволяет адаптировать процесс рассуждения под сложность вопроса и получать более точные и многогранные ответы.

Система ориентирована на исследовательские и прототипные задачи, демонстрируя возможность построения полноценного графового RAG на локальном оборудовании с открытыми моделями. Ключевые особенности — кэширование дорогостоящих операций, устойчивость к ошибкам модели, потокобезопасность при параллельной работе и автоматическое создание резервных копий графа при штатном завершении.

2 Архитектура системы

Центральным элементом является класс **GraphRAG**, который координирует работу всех подсистем. Основные компоненты и их взаимодействие таковы:

- **LLMClient** — обеспечивает обмен сообщениями с языковой моделью через OpenAI-совместимый HTTP-интерфейс. Поддерживает кэширование ответов в SQLite, автоматическую очистку старых записей и повторные попытки при сетевых ошибках.
- **EmbeddingClient** — вычисляет векторные представления текстов с помощью библиотеки sentence-transformers. Имеет два режима: одиночный и пакетный, с собственным кэш-слоем на SQLite.

- **MemgraphStore** — прослойка для работы с графовой базой данных Memgraph. Содержит методы для создания и удаления узлов и связей, управления транзакциями, векторного поиска и создания снейпшотов. Доступ к графу сериализуется замком (`threading.RLock`).
- **LLMEntityExtractor** и **RelationExtractor** — специализированные экстракторы, использующие LLM для извлечения сущностей и отношений из заданного текста. Оба наследуются от общего класса **BaseExtractor**, который содержит логику fallback-промптов и безопасного парсинга JSON. При неудачном разборе основного промпта применяется упрощённый.
- **ConflictResolver** — анализирует пары отношений с одинаковыми субъектом и объектом на предмет противоречий и разрешает их (удаление, оставление одного или слияние). Запускается вручную через метод `resolve_conflicts()`, а не автоматически при загрузке текста.
- **HistoryManager** — хранит историю диалога в оперативной памяти (циклическая очередь фиксированного размера) и предоставляет текстовое представление (в текущей версии история не используется при формировании промптов, но доступна для внешнего анализа).
- Модуль **got/** — реализует технику Graph of Thoughts на основе *Graph of Operations* (GoO). Включает классы **GoTController**, **GraphOfOperations**, **OperationNode**, операции **Generate**, **Score**, **Refine**, **Select**, **Merge**, а также вспомогательные **Prompter**, **Parser** и описание ролей в `roles.py`. Контроллер выполняет узлы графа в топологическом порядке, что обеспечивает гибкость и повторное использование мыслей.

Все компоненты слабо связаны: каждый модуль решает свою задачу и взаимодействует с другими через чётко определённые интерфейсы (методы и датаклассы Pydantic). Конфигурация системы вынесена в YAML-файл и загружается в датакласс **GraphRAGConfig**, что позволяет легко адаптировать систему под различное окружение.

Потоки данных при обработке текста: текст → чанки → для каждого чанка последовательно вызываются экстракторы (сущности → отношения) → результаты сохраняются в Memgraph в рамках одной транзакции на чанк. При ответе на вопрос выполняется обратный процесс: векторный поиск ближайших сущностей, извлечение их контекста (пассажи, соседние сущности) и формирование промпта для генерации ответа. Если включён GoT, перед финальным ответом запускается цепочка операций над «мыслями».

3 Модель графа знаний

Граф в Memgraph организован как трёхуровневая память, сочетающая схематические, фактологические и источниковедческие аспекты. Введём основные понятия.

Сущность (Entity) — это уникальный объект реального мира или абстракция, извлечённая из текста. Каждая сущность имеет имя (текст), тип (например, **PERSON**, **LOCATION**, **OBJECT**, **ACTION**, **TIME**) и векторное представление (embedding), вычисленное по её имени. Тип сущности связывается с узлом схемы (Schema) отношением

`INSTANCE_OF`, что позволяет в дальнейшем проводить семантическую фильтрацию и обобщение. Векторное представление — это плотный числовой вектор размерности, определяемой моделью эмбедингов (по умолчанию 384 для `all-MiniLM-L6-v2`), который отражает семантику имени сущности. Он используется для поиска близких по смыслу сущностей через векторный индекс.

Пассаж (Passage) — это исходный фрагмент текста (чанк), из которого были выделены сущности и отношения. Пассаж хранит полный текст и уникальный идентификатор (строка вида `<source>_<idx>` или `UUID`). Связь `MENTIONED_IN` соединяет сущность с пассажем, указывая, что данная сущность упоминалась в этом фрагменте. Это обеспечивает прослеживаемость: для любого факта можно найти его первоисточник. При повторном обнаружении той же сущности в том же пассаже связь не дублируется (используется `MERGE`).

Отношение (Relation) — направленная связь между двумя сущностями с меткой `RELATION` и атрибутами: `type` (глагольная фраза, например, «рубил», «растопила») и `based_on` (список идентификаторов пассажей, на основе которых было извлечено это отношение). Такой дизайн позволяет хранить множество доказательств для одного и того же факта: если отношение повторяется в разных чанках, их идентификаторы накапливаются в `based_on`, и при необходимости можно проследить все источники. При добавлении нового пассажа к существующему отношению используется условная конструкция в Cypher-запросе: `ON CREATE SET r.based_on = [$passage_id]; ON MATCH SET r.based_on = CASE WHEN $passage_id NOT IN r.based_on THEN r.based_on + [$passage_id] ELSE r.based_on END`. Это гарантирует отсутствие дубликатов и идемпотентность операций.

Схема (Schema) — узлы, описывающие типы сущностей (`entity_type`) и типы отношений (`relation_type`). Создаются автоматически при первом упоминании типа, что делает граф самоописывающим и облегчает интеграцию с внешними онтологиями.

Векторный индекс строится по полю `embedding` узлов `Entity` с использованием встроенного в Memgraph механизма на базе библиотеки `USearch`. При инициализации хранилища проверяется существование индекса с именем `entity_embeddings`; если размерность не совпадает с текущей моделью эмбедингов, индекс пересоздаётся. Это позволяет безболезненно менять модель эмбедингов, не разрушая граф.

4 Обработка входного текста

Метод `process_text` принимает произвольный текст и опциональный параметр `source`, возвращая список идентификаторов созданных пассажей. Разберём этапы подробно.

Чанкинг — процесс разбиения длинного текста на перекрывающиеся фрагменты (чанки) заданного размера (в символах). Перекрытие необходимо, чтобы не терять контекст на границах: отношения могут связывать сущности, расположенные в разных чанках, и без перекрытия часть связей могла бы быть упущена. Размер чанка и перекрытие задаются в конфигурации (по умолчанию 500 символов с перекрытием 100). Шаг сдвига вычисляется как `chunk_size - overlap`. Каждый чанк сохраняется в графе как узел `Passage` с уникальным идентификатором.

Далее для каждого чанка в отдельной транзакции выполняются следующие шаги:

- Включается временный кэш эмбедингов (на время обработки всех чанков), чтобы не вычислять повторно эмбединг для одинаковых имён сущностей.

- Вызывается `LLMEntityExtractor.extract`, который отправляет LLM промпт с просьбой извлечь сущности в формате JSON-массива объектов `{ "text": "...", "type": "..."}.` Для повышения надёжности сначала используется подробный промпт, а при неудачном разборе ответа (не удалось извлечь JSON) — упрощённый fallback-промпт. Ответ модели обрабатывается функцией `safe_parse_json`, которая умеет извлекать JSON даже из текста, обёрнутого в markdown-блоки или смешанного с пояснениями.
- Полученные сущности добавляются в граф через `store.add_entity`. Внутри метода: если сущность с таким именем уже существует, проверяется её тип (при необходимости обновляется с "UNKNOWN" на более конкретный), а связь с пассажем `MENTIONED_IN` создаётся операцией `MERGE`. Для новой сущности вычисляется эмбединг и создаётся связь со схемой.
- Затем `RelationExtractor.extract` принимает список извлечённых сущностей и текст чанка, формирует промпт с перечислением сущностей и получает от модели JSON-массив троек `head, relation, tail`. Здесь также используется fallback-промпт. Каждое отношение валидируется через Pydantic и добавляется в граф с идентификатором текущего пассажа.
- При успешном завершении всех операций транзакция коммитится; если на любом этапе возникло исключение — транзакция откатывается, и чанк пропускается. Это обеспечивает атомарность: граф не загрязняется частично обработанными данными.

После обработки всех чанков временный кэш эмбедингов отключается. Метод не вызывает автоматически ни разрешение конфликтов, ни создание снэпшота. Конфликты можно разрешить отдельным вызовом `resolve_conflicts()`, а снэпшот создаётся при явном закрытии экземпляра или при завершении процесса (см. секцию «Хранилище Memgraph»).

Все метрики (количество сущностей, отношений, пассажей и время обработки) накапливаются в атрибуте `metrics` экземпляра `GraphRAG`, что позволяет отслеживать производительность и объём извлечённых знаний.

5 Разрешение конфликтов

Проблема конфликтов возникает из-за того, что разные чанки могут описывать одни и те же сущности, но приписывать им противоречивые действия. Например, в одном фрагменте «Иван купил молоко», а в другом «Иван продал молоко». Оставлять оба отношения в графе нельзя — они противоречат друг другу и сделают дальнейшие ответы несогласованными. Задача конфликт-резольвера — обнаружить такие пары и принять решение: оставить одно, удалить оба и заменить на новое (слияние) или, если конфликта нет, сохранить оба. Этап запускается вручную методом `resolve_conflicts(dry_run=False)` и может быть выполнен в режиме «dry run» без изменения графа.

5.1 Обнаружение конфликтов

Класс `ConflictResolver` агрегирует все отношения, сгруппированные по паре (`head`, `tail`). Для каждой группы, где больше одного отношения, выполняется попарное срав-

нение всех комбинаций с разными названиями. Для каждой пары извлекается текст первого связанного пассажира (по первому идентификатору из списка `based_on`), после чего формируется специализированный промпт:

Ты — система разрешения конфликтов в графе знаний. Даны две тройки (субъект, отношение, объект) ... Определи, являются ли эти отношения конфликтующими ... Верни ТОЛЬКО JSON с полями: `conflict`, `resolution`, `new_relation`, `explanation`.

Модель должна вернуть JSON, где `resolution` принимает одно из трёх значений: `"keep_first"`, `"keep_second"` или `"merge"` (при `conflict=true`). Если ответ не удаётся распарсить или значение `resolution` некорректно, выполняется до `max_retries` повторных попыток. При окончательной неудаче конфликт логируется, но граф не изменяется — система продолжает работу с оставшимися отношениями.

5.2 Итеративное разрешение

Ключевой методологический нюанс — применение решений. Текущая реализация использует итеративный подход внутри каждой группы (`_resolve_group`).

Алгоритм для одной пары (`head`, `tail`):

1. Строится рабочий список отношений — копий словарей с `relation` и `based_on`.
2. Пока в списке больше одного элемента:
 - (a) Для каждой несовпадающей пары вызывается `_get_resolution`.
 - (b) Если конфликт обнаружен:
 - `"keep_first"` — удаляется второе отношение из графа и из рабочего списка.
 - `"keep_second"` — удаляется первое.
 - `"merge"` — оба исходных отношения удаляются, создаётся новое с именем `new_relation` (или «связан» по умолчанию). В его `based_on` попадают все идентификаторы пассажиров обоих старых отношений. В рабочем списке вместо двух удалённых появляется одно новое отношение.
 - (c) После любого изменения списка (добавления/удаления) цикл начинается заново: повторно перебираются все пары с новым составом.
3. Если за полный проход не найдено ни одного конфликта, итерации прекращаются.

Такой подход гарантирует, что каждое последующее решение принимается с учётом уже выполненных изменений, и граф остаётся консистентным. При слиянии особенно важно сохранить все ссылки на пассажиры, чтобы новое отношение имело полную доказательную базу.

6 Вопросно-ответная система

Основной метод `ask` реализует два режима: простой (без Graph of Thoughts) и с Graph of Thoughts. Рассмотрим их подробно.

6.1 Простой режим (use_got=False)

При отключённом GoT вызывается метод `_direct_answer`. Его шаги:

- Векторный поиск: пользовательский вопрос преобразуется в эмбединг, и в графе ищутся ближайшие сущности (по косинусному расстоянию через векторный индекс). Количество результатов ограничено параметром `top_k`.
- Для каждой найденной сущности извлекаются связанные отношения с текстами пассажиров. Если включён случайный обход (`random walk`), дополнительно собираются соседние сущности. Механизм обхода: для стартовой сущности проходят по цепочкам отношений до заданной глубины и выбирают ограниченное число соседей.
- Все собранные фрагменты контекста объединяются и обрезаются до `max_context_length` символов.
- Формируется промпт, включающий контекст и вопрос. Ответ генерируется LLM.

6.2 Graph of Thoughts (use_got=True)

Техника Graph of Thoughts (GoT) в данной реализации базируется на *Graph of Operations* (GoO) — направленном ациклическом графе, узлами которого являются операции над мыслями, а рёбра определяют зависимости. Это позволяет гибко строить цепочки рассуждений, комбинировать различные стратегии и повторно использовать промежуточные результаты. В отличие от линейных пайплайнов, GoO даёт возможность ветвления, слияния и итеративного улучшения мыслей в произвольном порядке.

При вызове метода `ask` с параметром `use_got=True` система динамически строит граф операций на основе параметров конфигурации. Стандартный граф включает следующие узлы:

1. **Генерация (Generate)** — создаётся `got_num_thoughts` (по умолчанию 3) исходных мыслей. Каждая мысль генерируется с уникальной ролью (Аналитик, Критик, Стратег, Скептик, Учёный), что обеспечивает разнообразие точек зрения. Роли могут выбираться случайно или по порядку (`got_use_random_roles`). Температура генерации задаётся параметром `got_generation_temperature`. Входные данные для этого узла — контекст, полученный из графа знаний, и вопрос пользователя.
2. **Оценка (Score)** — каждая мысль оценивается LLM по шкале от 0 до 10. Промпт запрашивает только числовое значение. Парсер `parse_score` извлекает первую числовую оценку из ответа. Оценка сохраняется в объекте мысли и используется на следующих этапах.
3. **Селекция (Select)** — опциональный узел, который оставляет только `got_select_top_k` мыслей с наивысшей оценкой. Этот этап позволяет отсеять слабые варианты и сконцентрировать вычислительные ресурсы на наиболее перспективных рассуждениях. Включается параметром `got_use_select`.
4. **Улучшение (Refine)** — для каждой отобранной мысли выполняется улучшение. Промпт просит модель сделать ответ более точным, полным и логичным.

Этот узел может повторяться несколько раз; количество итераций задаётся параметром `got_iterations`. Температура улучшения — `got_refine_temperature`. При каждой итерации может быть повторно вызвана оценка, если включён параметр `got_score_enabled`.

5. **Слияние (Merge)** — финальный узел, выполняемый, если `got_merge_enabled` и число мыслей больше 1. Все итоговые мысли объединяются в одном промпте, и модель синтезирует единый связный ответ, вбирающий лучшие стороны каждого варианта. Температура слияния — `got_merge_temperature`.

После выполнения графа итоговый ответ сохраняется в историю диалога через `HistoryManager`. В текущей версии история не подставляется в промпты следующих вопросов, но может быть использована для внешнего логирования или последующего развития системы.

7 Хранилище Memgraph

7.1 Транзакции и блокировки

Все публичные методы `MemgraphStore` защищены одним общим замком (`threading.RLock`). Это решение продиктовано тем, что стандартный клиент `GraphQLAlchemy` не гарантирует потокобезопасность: параллельные запросы из разных потоков могут перемешиваться, нарушая целостность операций. Блокировка захватывается перед выполнением любого запроса к базе и освобождается после его завершения, что сериализует доступ. Хотя это потенциально снижает параллелизм при интенсивной работе с графом, для прототипа такой подход оправдан, поскольку основные задержки приходятся на LLM, а не на операции с данными.

Транзакции управляются явно методами `begin`, `commit` и `rollback`. При обработке чанка все операции с данным чанком заключаются в одну транзакцию на уровне метода `process_text`. Метод `add_entity` сам транзакций не открывает; он выполняется внутри уже запущенной транзакции или (вне обработки текста) в режиме автокоммита отдельных запросов.

7.2 Работа с пассажирами

Каждый пассаж — это узел `Passage` с полем `text` и уникальным `id`. При добавлении пассажира используется `MERGE`, что гарантирует идемпотентность. Связь сущности с пассажиром (`MENTIONED_IN`) также создаётся через `MERGE`, чтобы предотвратить появление множественных рёбер при многократном вызове `add_entity` для одной и той же пары (сущность, пассаж).

7.3 Отношения и `based_on`

Отношения представлены рёбрами типа `RELATION` с атрибутом `type` и списком `based_on`. При добавлении отношения с новым пассажиром выполняется `MERGE` с условной логикой:

- Если ребро создаётся впервые (`ON CREATE`), `based_on` инициализируется списком из одного элемента — текущего `passage_id`.

- Если ребро уже существует (**ON MATCH**), проверяется, нет ли уже такого `passage_id` в списке; если нет — он добавляется.

Такой подход обеспечивает, что каждое отношение хранит все источники, из которых оно было извлечено, и при этом не замусоривается дубликатами.

7.4 Векторный поиск

Метод `vector_search` вычисляет эмбединг запроса с помощью `EmbeddingClient`, затем вызывает встроенную процедуру `Memgraph vector_search.search`, передавая имя индекса, желаемое количество результатов и вектор. Процедура возвращает узлы и расстояния; результаты преобразуются в список словарей с именем, типом и `score`. Индекс использует библиотеку `USearch`, которая обеспечивает быстрое приближённое нахождение ближайших соседей, критичное для скорости ответа.

7.5 Снепшоты и очистка

Снепшоты `Memgraph` — это резервные копии всего состояния графа на диск. Метод `snapshot` вызывается автоматически при явном вызове `close()` экземпляра `GraphRAG` и при аварийном завершении процесса (через глобальный обработчик `atexit`). Механизм `graceful shutdown` регистрирует все активные экземпляры и гарантирует, что даже при неожиданной остановке (например, по `Ctrl+C`) данные не будут потеряны. Сам процесс загрузки текста снепшот не создаёт, оставляя это на усмотрение пользователя или завершающего этапа.

Метод `clear` полностью очищает граф: удаляет все узлы и связи командой `MATCH (n) DETACH DELETE n`. Если использовался векторный индекс, он также удаляется (`DROP VECTOR INDEX entity_embeddings`) и затем немедленно создаётся заново в блоке `finally`, что гарантирует восстановление индекса даже при ошибке удаления. Это позволяет быстро подготовить чистое окружение для нового эксперимента без ручного вмешательства. Также `clear` сбрасывает метрики и историю диалога.

8 Клиенты LLM и эмбедингов

8.1 LLMClient

Взаимодействие с языковой моделью построено поверх библиотеки `openai` в режиме совместимости с локальными серверами. Клиент принимает объект `LLMConfig` (модель, URL, ключ, температура) и путь к файлу SQLite-кэша. Кэширование реализовано через таблицу `cache` с составным ключом, образованным MD5-хешем от конкатенации промпта, имени модели и температуры, и полями `response` и `timestamp`. Учёт температуры в ключе предотвращает смешивание ответов, полученных при разных настройках креативности. При запросе сначала проверяется кэш; при попадании ответ возвращается мгновенно. При промахе выполняется HTTP-вызов, и успешный ответ сохраняется с текущей временной меткой.

Для повышения отказоустойчивости реализован механизм повторных попыток: при сетевой ошибке или таймауте делается до `max_retries` попыток с экспоненциальной задержкой (начиная с `retry_delay` секунд, умноженной на 2^{attempt}). Кэш можно

периодически чистить от устаревших записей методом `cleanup_cache` (по умолчанию удаляются ответы старше 1 часа), что предотвращает неконтролируемый рост базы.

При создании клиента автоматически проверяется структура таблицы, и если отсутствует колонка `timestamp` (например, после миграции со старой версии), она добавляется. Это обеспечивает обратную совместимость.

8.2 EmbeddingClient

Для вычисления текстовых эмбеддингов используется библиотека `sentence-transformers` с моделью по умолчанию `all-MiniLM-L6-v2` (размерность 384). Модель загружается лениво при первом обращении к свойству `model`, что экономит память до момента реальной необходимости. Кэширование эмбеддингов реализовано в отдельной таблице `embedding_cache` с ключом на основе MD5-хеша от текста и опциональных параметров `prompt_name` и `prompt`.

Два режима работы:

- Одиночный (`embed`) — для одного текста: при промахе кодирует текст с нормализацией и точностью `float32`, сохраняет в кэш и возвращает.
- Пакетный (`embed_batch`) — для списка текстов: сначала проверяется кэш для каждого; отсутствующие собираются и кодируются одним вызовом `model.encode` с параметром `batch_size`. Затем результаты сохраняются в кэш. Вся работа с базой в этом методе ведётся в одном соединении, что снижает накладные расходы.

Пакетная обработка существенно ускоряет вычисление эмбеддингов для множества сущностей, так как используется преимущество векторизованных вычислений модели и сокращается количество раундов обращения к базе. Кроме того, во время массовой загрузки текста включается временный кэш в оперативной памяти (`enable_batch_cache/disable_batch_cache`), который сохраняет эмбеддинги сущностей в словаре, устраняя повторные обращения к базе и модели для одинаковых имён.

9 Утилиты безопасного парсинга JSON

Одной из проблем при использовании LLM является нестабильность формата ответа. Модель может добавить пояснительный текст до или после JSON, обернуть его в markdown-разметку (“`json ...`”) или вовсе выдать некорректный JSON. Функция `safe_parse_json` реализует устойчивый подход:

1. Ищет в ответе блок, начинающийся с “`json`” и заканчивающийся “`”`”. Извлекает содержимое.
2. Если блок не найден, ищет первую открывающую скобку (`[` или `{`) и соответствующую последнюю закрывающую. Это позволяет извлечь JSON, даже если вокруг него есть другой текст.
3. Пытается разобрать полученную строку с помощью `json.loads`. При неудаче возвращает `None`.

Такой эвристический подход покрывает подавляющее большинство реальных ответов современных LLM (Qwen, Llama, Mistral) и делает пайплайн значительно более надёжным без необходимости тонкой настройки модели.

10 Особенности реализации и принятые решения

В этом разделе сведены ключевые технические и методологические решения с акцентом на их взаимосвязь.

- **Потокобезопасность.** Блокировка в `MemgraphStore` (рекурсивный замок) сериализует доступ к графу, а сам GoT выполняется последовательно, поэтому конфликтов на уровне HTTP-клиента не возникает. Это компромисс между производительностью и простотой реализации.
- **Graceful shutdown.** Глобальная регистрация экземпляров и вызов `snapshot()` через `atexit` гарантируют сохранность данных даже при аварийном завершении. Явное закрытие экземпляра освобождает ресурсы и удаляет его из списка. Снепшот не создаётся автоматически после каждого `process_text`, чтобы не нагружать диск при частых экспериментах.
- **Устойчивость к ошибкам модели.** Сочетание fallback-промттов, `safe_parse_json` и повторных попыток с экспоненциальной задержкой позволяет системе работать даже с нестабильными или слабыми LLM.
- **Итеративное разрешение конфликтов.** Отказ от предварительного сбора всех конфликтов в пользу итеративного перестроения рабочего списка предотвращает ошибки, связанные с удалением уже удалённых объектов. Конфликт-резолювер не привязан жёстко к загрузке текста и может применяться выборочно.
- **Кэширование на всех уровнях.** LLM-запросы кэшируются по полному промπτу с учётом температуры, эмбединга — по тексту. Временный оперативный кэш при загрузке текста дополнительно экономит вычислительные ресурсы. Это ускоряет повторные запуски и отладку.
- **Атомарность операций с чанком.** Каждый чанк обрабатывается в рамках одной транзакции: либо все извлечённые данные попадают в граф, либо ни одно. Это предотвращает «грязное» состояние при частичных сбоях.
- **Восстановление индекса при очистке.** Даже если удаление старого индекса не удалось, новый индекс гарантированно создаётся в блоке `finally`, что сохраняет работоспособность векторного поиска.
- **Ролевые модели в GoT.** Использование ролей (Аналитик, Критик, Стратег, Скептик, Учёный) при генерации мыслей вносит разнообразие и позволяет получить ответы, учитывающие разные аспекты вопроса. Случайный выбор без повторений по умолчанию исключает дублирование точек зрения.
- **Гибкая конфигурация.** Параметры системы вынесены в YAML, что позволяет адаптировать её под разные окружения и модели без изменения кода.

- **Graph of Operations (GoO) как основа GoT.** В отличие от линейных пайплайнов, граф операций позволяет строить произвольные цепочки рассуждений с ветвлениями и слияниями. Это даёт возможность адаптировать процесс генерации под конкретную задачу: например, для простых вопросов можно выполнить только **Generate** и **Score**, а для сложных — добавить **Select**, несколько итераций **Refine** и финальное **Merge**. Такая гибкость достигается за счёт явного описания зависимостей между узлами и топологической сортировки при выполнении.

Принятые решения делают систему надёжной, гибкой и удобной для экспериментов, не перегружая её излишней сложностью.

11 Заключение

Представленная система демонстрирует полный цикл построения графа знаний из неструктурированного текста и последующего использования этого графа для ответа на вопросы. Архитектура модульная, что упрощает замену отдельных компонентов (например, переход на другую LLM или графовую СУБД). Особое внимание уделено практическим аспектам: кэшированию дорогих операций, обработке ошибок модели, потокобезопасности и сохранности данных.

Проект может служить основой для исследовательских прототипов в области NLP и графовых баз данных.